

Kamień Milowy II – Implementacja systemu

Aplikacja do zarządzania zadaniami

Bartłomiej Karkoszka

Rafał Grot

Filip Stępień

Przedmiot: Modelowanie i Analiza Systemów Informatycznych

Grupa: 1ID21A

Spis treści

1	Wprowadzenie	2
2	Stan implementacji	2
2.1	Struktura repozytorium	2
2.2	Szczegóły implementacyjne	2
2.3	Lista endpointów REST	3
2.4	Schemat bazy danych	3
3	Zaktualizowane diagramy z kamienia I	3
3.1	Diagram przypadków użycia	3
3.2	Diagram klas	4
3.3	Diagram komponentów	5
3.4	Diagram warstw systemu	7
4	Diagram sekwencji – utworzenie zadania	7
5	Diagram aktywności – zarządzanie zadaniem	9
6	Uruchamianie i weryfikacja	9
6.1	Wymagania wstępne	9
6.2	Kroki uruchomienia	10
6.3	Weryfikacja	10
7	Podsumowanie	10

1 Wprowadzenie

Niniejszy dokument jest **uzupełnieniem** raportu z kamienia milowego I i koncentruje się wyłącznie na elementach wymaganych w drugim etapie: stanie implementacji, diagramie sekwencji, diagramie aktywności oraz aktualizacjach wcześniej przygotowanych diagramów. Pozostałe sekcje (cel i zakres systemu, aktorzy, wymagania funkcjonalne i niefunkcjonalne, słownik pojęć, pełen stos technologiczny) nie są tu powtarzane – pozostają one w dokumencie KM I.

W stosunku do KM I nastąpiła jedna istotna zmiana stosu technologicznego: zaplanowany Spring Boot (Java) po stronie backendu został zastąpiony przez **Go** (router **chi**, generator **sqlc**). Decyzja była motywowana chęcią wypróbowania języka Go w praktyce. Pozostałe elementy stosu (frontend React, Keycloak, PostgreSQL, Docker) pozostały bez zmian.

2 Stan implementacji

2.1 Struktura repozytorium

Projekt zorganizowany jest jako monorepo zawierające dwie aplikacje:

- `apps/task-api` – backend w Go (REST API),
- `apps/task-frontend` – frontend w React 19 (SPA),

oraz konfigurację infrastruktury (`docker-compose.yml`, `docker/keycloak/realm.json`, `flake.nix`).

2.2 Szczegóły implementacyjne

Poniżej zebrano elementy implementacyjne uzupełniające wymagania funkcjonalne z KM I:

- **AuthMiddleware (chi)** – weryfikuje JWT w oparciu o klucze publiczne pobierane z JWKS Keycloak; przy pierwszym żądaniu nowego użytkownika automatycznie tworzy wpis w tabeli `users` na podstawie pól `sub`, `name`, `email` z tokenu.
- **Walidacja DTO** – metody `Validate()` na strukturach żądania kontrolują obecność tytułu, dopuszczalne wartości statusu/priorytetu oraz format daty (YYYY-MM-DD).
- **Generowane zapytania (sqlc)** – wszystkie operacje na bazie korzystają ze statycznie typowanych funkcji wygenerowanych z plików `queries/*.sql`; agregacja dashboardu wykonywana jest jednym zapytaniem `GROUP BY status`.
- **Swagger UI** – automatycznie generowana specyfikacja OpenAPI dostępna pod `/swagger`, z konfiguracją OAuth2/PKCE wskazującą na realm Keycloak.
- **Frontend (Redux Toolkit)** – stan zadań trzymany w `slice` z `createEntityAdapter`, operacje wykonywane przez `thunki` (`fetchTasks`, `createTask`, `deleteTask`).

2.3 Lista endpointów REST

Wszystkie endpointy są zabezpieczone middleware **Protect** (wymagany nagłówek **Authorization: Bearer <JWT>**) i udostępnione pod prefiksem **/api** (konfigurowalnym przez zmienną **API_BASE_PATH**).

Metoda	Ścieżka	Opis	Kody
GET	/health	Stan aplikacji i bazy danych	200, 401, 500
GET	/users/{userId}/tasks	Lista zadań użytkownika z opcjonalnymi filtrami	200, 400, 401, 500
POST	/users/{userId}/tasks	Utworzenie nowego zadania	201, 400, 401, 500
GET	/users/{userId}/tasks/{id}	Szczegóły pojedynczego zadania	200, 400, 401, 404, 500
PUT	/users/{userId}/tasks/{id}	Aktualizacja zadania	200, 400, 401, 404, 500
DELETE	/users/{userId}/tasks/{id}	Usunięcie zadania	204, 400, 401, 404, 500
GET	/users/{userId}/dashboard	Statystyki zadań użytkownika	200, 400, 401, 500

2.4 Schemat bazy danych

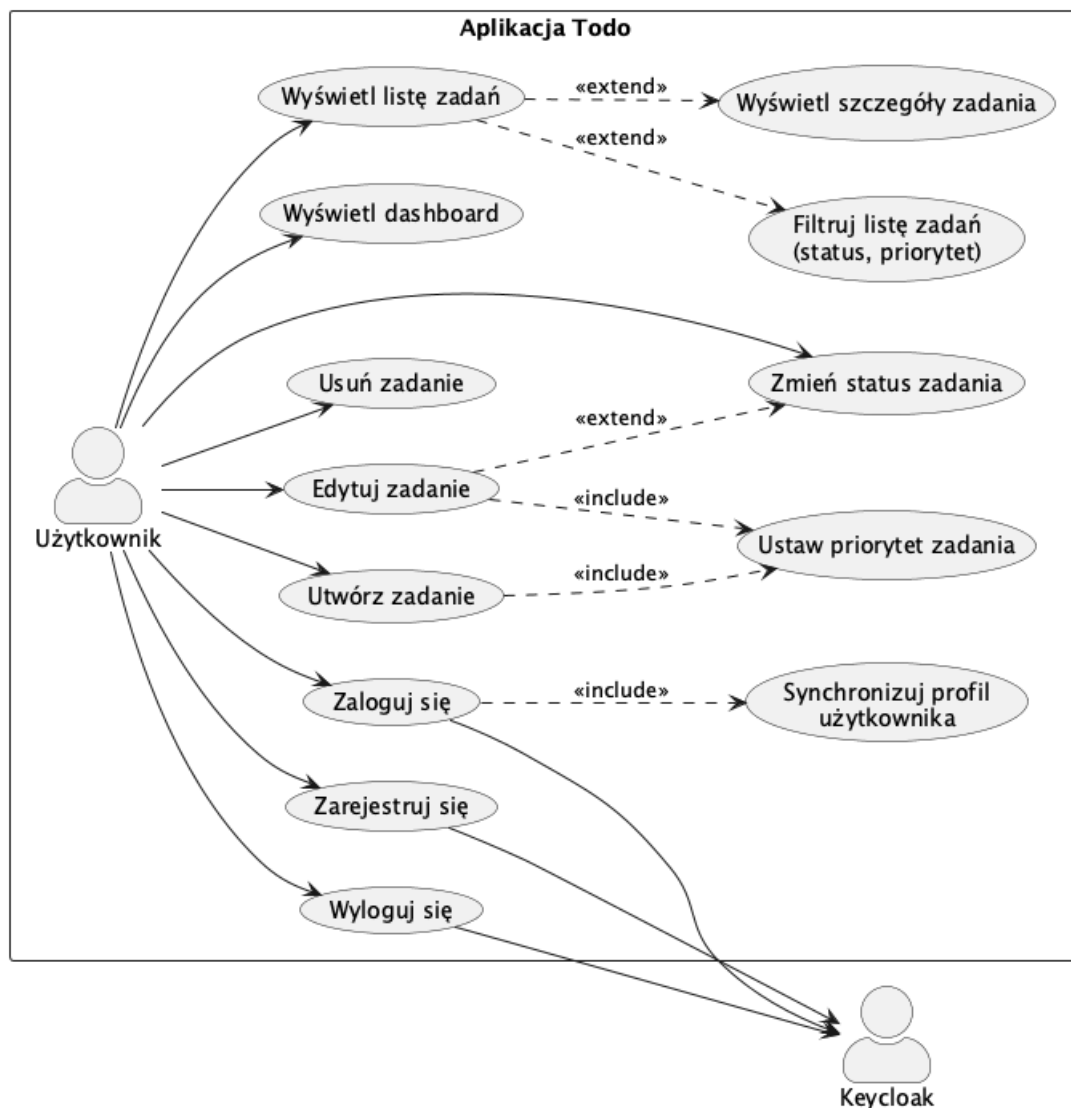
Migracje (goose) tworzą tabele **users** oraz **tasks**, a także typy wyliczeniowe **task_status** (**todo**, **in_progress**, **done**) i **task_priority** (**low**, **medium**, **high**). Tabela **tasks** posiada klucz obcy **user_id** z kaskadowym usuwaniem oraz znaczniki czasowe **created_at**, **updated_at**.

3 Zaktualizowane diagramy z kamienia I

Wszystkie diagramy zostały dostosowane do rzeczywistej implementacji.

3.1 Diagram przypadków użycia

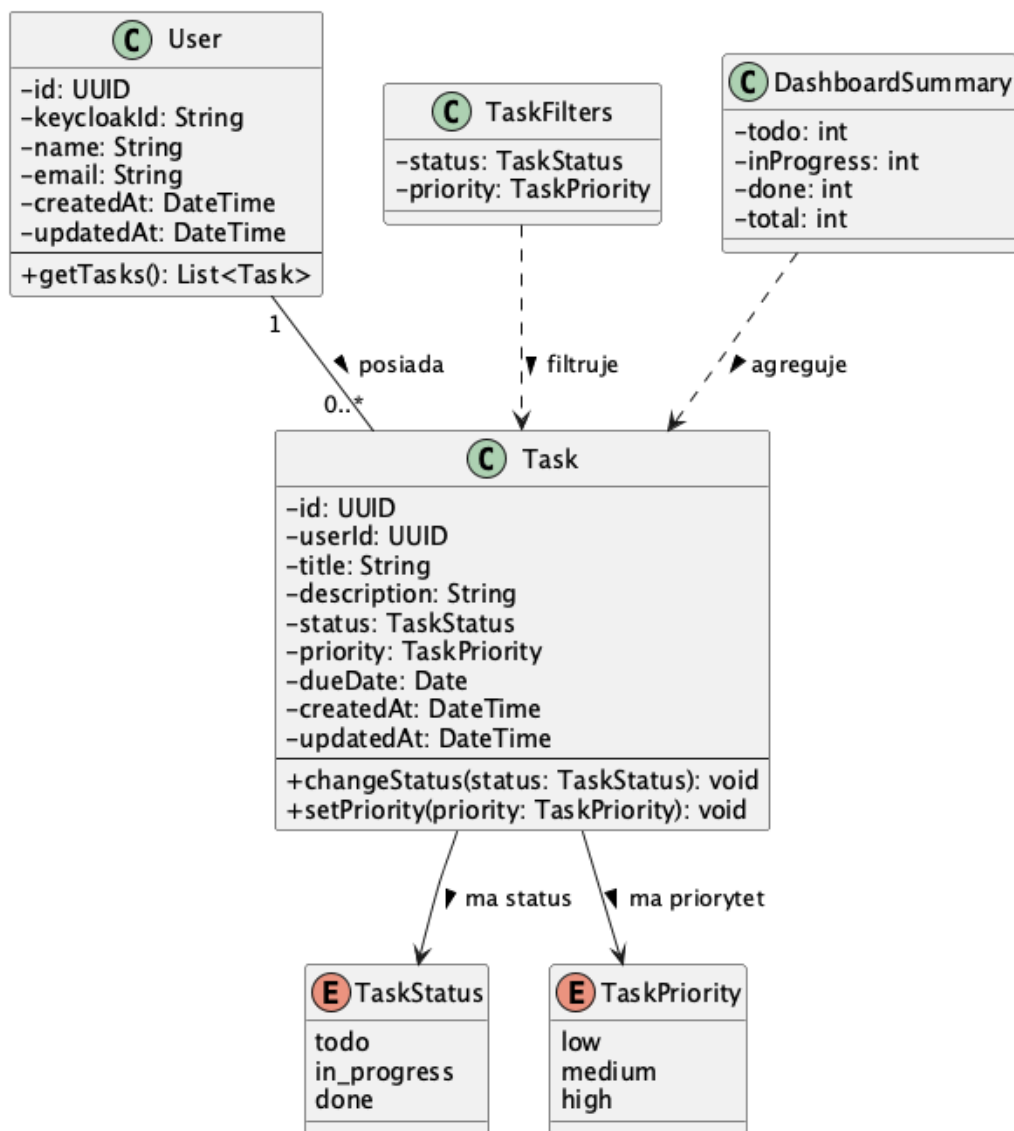
Diagram został otoczony granicą systemu *Aplikacja Todo. Użytkownik* – jako bezpośredni odbiorca aplikacji – został umieszczony wewnątrz granicy, natomiast *Keycloak* jako zewnętrzny dostawca uwierzytelniania pozostaje poza nią. Dodano przypadek *Synchronizuj profil użytkownika* (<<include>> w *Zaloguj się*) odpowiadający faktycznemu mechanizmowi tworzenia rekordu w tabeli **users** przy pierwszym żądaniu autoryzowanym.



Rysunek 1: Diagram przypadków użycia

3.2 Diagram klas

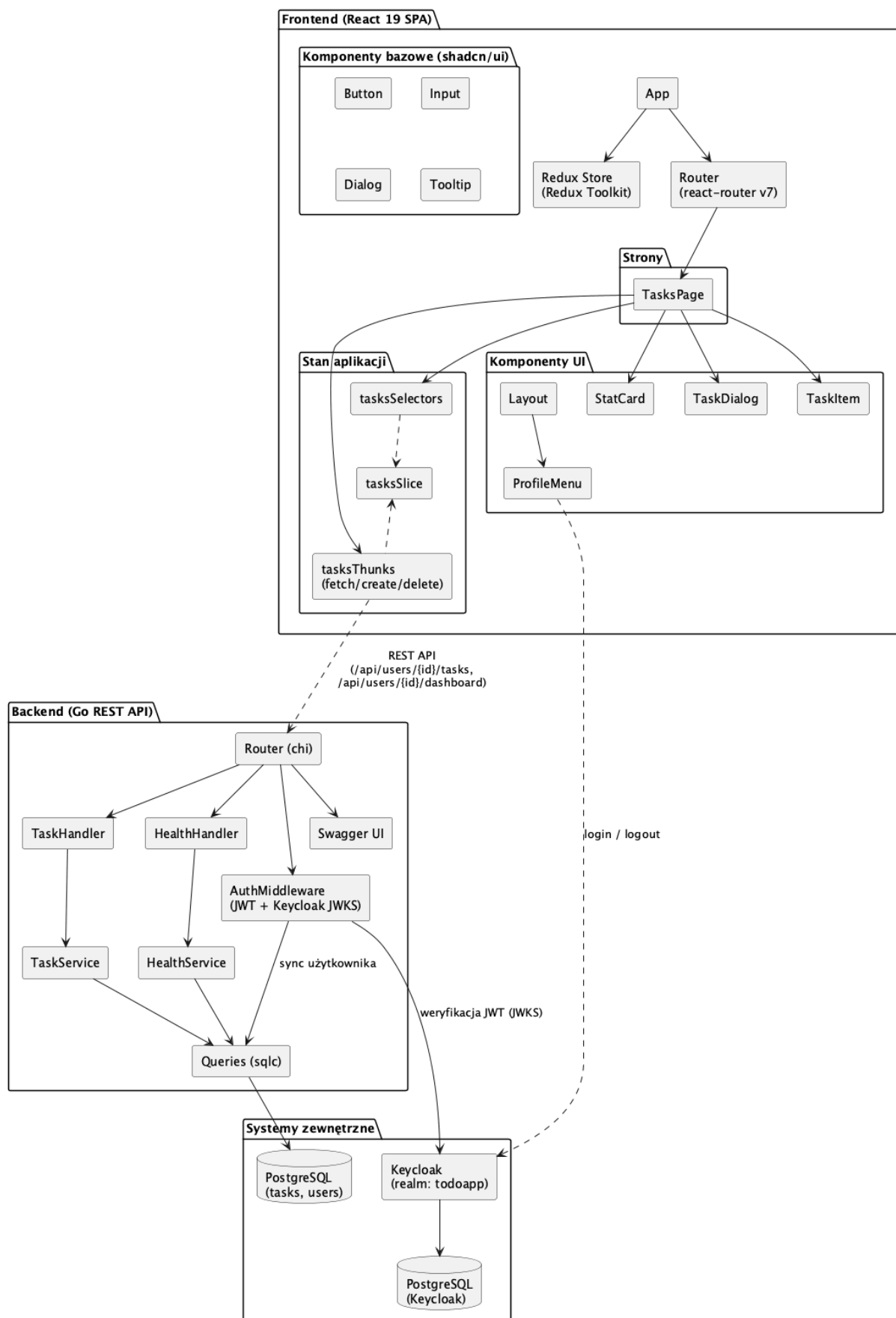
Klasa `User` została rozszerzona o pola `name`, `email`, `createdAt`, `updatedAt`, zgodnie z migracją `00002_add_user_profile_columns.sql`. Klasa `Task` otrzymała pole `userId` (klucz obcy). Wartości typów wyliczeniowych dostosowano do nazw rzeczywistych (`todo`, `in_progress`, `done`; `low`, `medium`, `high`). Dodano klasy pomocnicze `TaskFilters` oraz `DashboardSummary` odpowiadające DTO obecnym w kodzie.



Rysunek 2: Diagram klas (model dziedziny + DTO)

3.3 Diagram komponentów

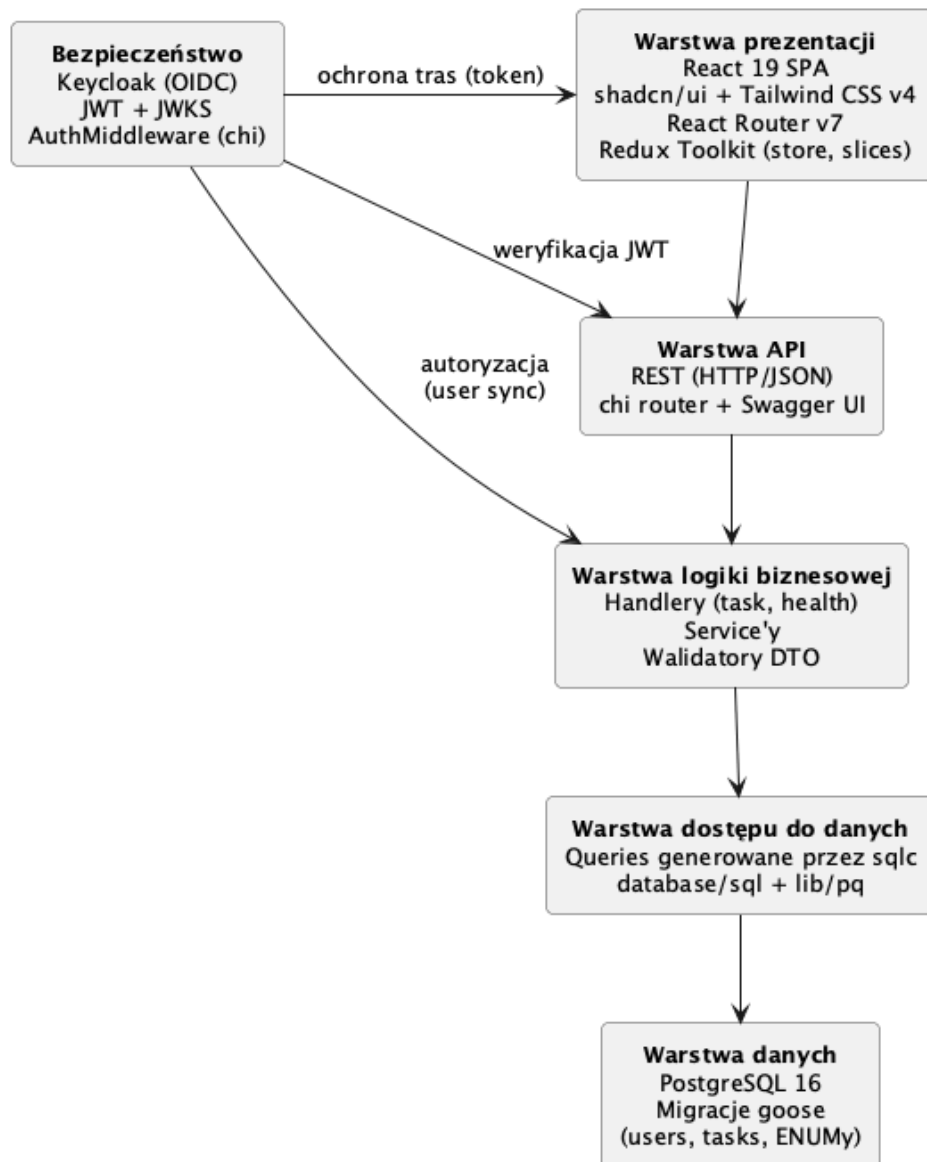
Diagram odzwierciedla rzeczywiste pakiety: po stronie frontendu wyodrębniono Redux Store (slice, thunki, selektory) oraz konkretne komponenty UI (Layout, TaskDialog, TaskItem, StatCard, ProfileMenu). Po stronie backendu przedstawiono router `chi`, `AuthMiddleware` z weryfikacją JWKS, handlery `TaskHandler` i `HealthHandler`, warstwę serwisów oraz wygenerowane przez `sqlc` Queries. Endpoint Swagger UI został wskazany jako osobny komponent.



Rysunek 3: Diagram komponentów

3.4 Diagram warstw systemu

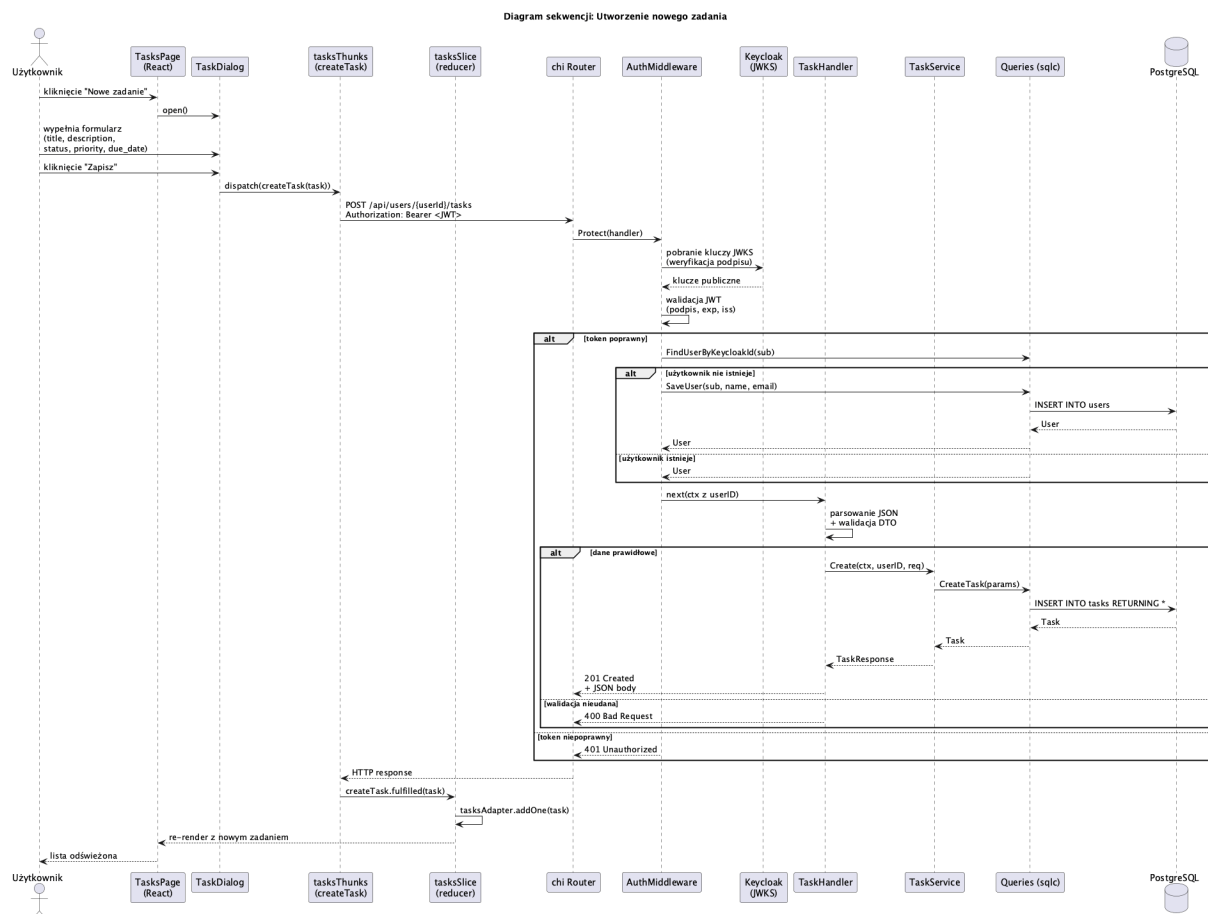
Etykiety warstw zaktualizowano zgodnie z zaimplementowanymi technologiami: warstwa prezentacji opisuje stack React 19 + Redux Toolkit, warstwa API – chi i Swagger UI, warstwa logiki – handlers i serwisy, warstwa dostępu do danych – generowane przez sqlc zapytania, a warstwa bezpieczeństwa – middleware weryfikujący JWT.



Rysunek 4: Diagram warstw systemu

4 Diagram sekwencji – utworzenie zadania

Wybrany scenariusz: użytkownik tworzy nowe zadanie. Scenariusz został wybrany, ponieważ ilustruje pełny przepływ przez wszystkie warstwy systemu – od interakcji w UI, przez akcję Redux i żądanie HTTP, weryfikację tokenu, synchronizację użytkownika, walidację DTO, persystencję w bazie, aż po aktualizację stanu po stronie klienta.



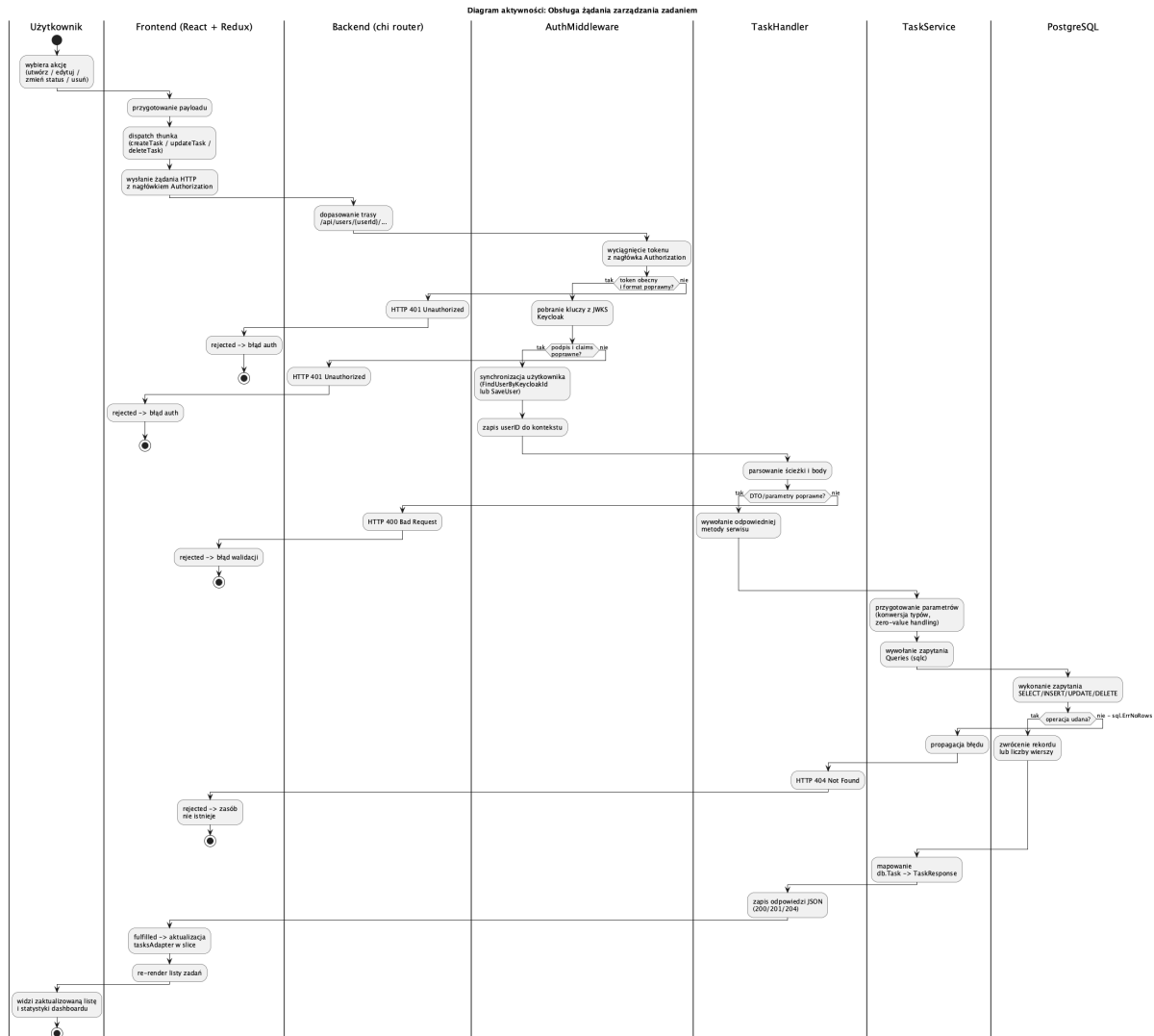
Rysunek 5: Diagram sekwencji: utworzenie nowego zadania

Opis kluczowych kroków.

1. Użytkownik otwiera dialog **TaskDialog** ze strony **TasksPage** i zatwierdza formularz.
2. Frontend wywołuje thunk **createTask**, który wysyła żądanie **POST /api/users/{userId}/tasks** wraz z tokenem JWT.
3. **AuthMiddleware** pobiera klucze z JWKS Keycloak i weryfikuje podpis tokenu.
4. Jeżeli użytkownik o podanym **sub** nie istnieje w bazie, jest tworzony (synchronizacja profilu); w przeciwnym wypadku rekord jest tylko odczytywany.
5. **TaskHandler** parsuje ciało żądania, uruchamia **Validate()** na DTO, a następnie wywołuje **TaskService.Create**.
6. Serwis przekazuje parametry do wygenerowanego zapytania **CreateTask**, które wykonuje **INSERT INTO tasks RETURNING ***.
7. Odpowiedź **201 Created** z zserializowanym zasobem trafia do thunka, który wywołuje akcję **createTask.fulfilled** – reducer dodaje encję do **tasksAdapter**, co skutkuje ponownym renderowaniem listy.

5 Diagram aktywności – zarządzanie zadaniem

Wybrany proces: ogólna obsługa żądania zarządzania zadaniem (utworzenie, edycja, zmiana statusu, usunięcie). Diagram zawiera punkty decyzyjne dla wszystkich istotnych ścieżek błędów obecnych w implementacji: 401 (brak/niepoprawny token), 400 (błędne dane wejściowe) oraz 404 (`sql.ErrNoRows`).



Rysunek 6: Diagram aktywności: obsługa żądania zarządzania zadaniem

Tory (swimlanes). Diagram wyodrębnia odpowiedzialności komponentów: *Użytkownik*, *Frontend (React + Redux)*, *Backend (chi router)*, *AuthMiddleware*, *TaskHandler*, *TaskService* oraz *PostgreSQL*.

6 Uruchamianie i weryfikacja

6.1 Wymagania wstępne

Docker, Docker Compose, Go 1.22+, Node.js 20+ oraz **task** (Taskfile).

6.2 Kroki uruchomienia

1. Skopiowanie pliku `.env.example` do `.env` i uzupełnienie zmiennych.
2. Start usług infrastrukturalnych: `docker compose up -d postgres postgres-keycloak keycloak adminer`.
3. Wykonanie migracji bazy danych: `task migrate` (lub `goose`).
4. Uruchomienie API: `task api:dev` (`go run ./cmd`).
5. Uruchomienie frontendu: `npm run dev` w katalogu `apps/task-frontend`.

6.3 Weryfikacja

Po starcie systemu dostępne są:

- Swagger UI – `http://localhost:<API_PORT>/swagger/index.html`,
- panel Keycloak – `http://localhost:8080`,
- Adminer – `http://localhost:8081`,
- frontend – `http://localhost:5173`.

7 Podsumowanie

W ramach drugiego kamienia milowego dostarczono działającą wersję systemu obejmującą podstawowe scenariusze użycia: uwierzytelnianie przez Keycloak, zarządzanie zadaniami (CRUD), filtrowanie oraz dashboard. Dokumentacja projektowa została rozszerzona o dwa nowe diagramy (sekwencji oraz aktywności), a wcześniej przygotowane diagramy zaktualizowano zgodnie z rzeczywistą implementacją – w tym kluczową zmianą stosu backendowego (Go zamiast Spring Boot) oraz rozszerzonym modelem użytkownika.